

汇编指令大全

一、数据传输指令

1. 通用数据传送指令

MOV: 传送字或字节。例: MOV AX, BX 将 BX 的值传送到 AX。

MOVSX: 先符号扩展, 再传送。例: MOVSX EAX, AL 将 AL 的符号扩展传送到 EAX。

MOVZX: 先零扩展, 再传送。例: MOVZX EDX, CX 将 CX 的零扩展传送到 EDX。

PUSH: 把字压入堆栈。例: PUSH BX 将 BX 的值压入堆栈。

POP: 把字弹出堆栈。例: POP DX 将堆栈顶部的值弹出到 DX。

PUSHA: 把 AX, CX, DX, BX, SP, BP, SI, DI 依次压入堆栈。例: PUSHA。

POPA: 把 DI, SI, BP, SP, BX, DX, CX, AX 依次弹出堆栈。例: POPA。

PUSHAD: 把 EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI 依次压入堆栈。例: PUSHAD。

POPAD: 把 EDI, ESI, EBP, ESP, EBX, EDX, ECX, EAX 依次弹出堆栈。例: POPAD。

BSWAP: 交换 32 位寄存器里字节的顺序。例: BSWAP EAX。

2. 输入输出端口传送指令

IN: I/O 端口输入。语法: IN AL, 0x60 从端口 0x60 读取数据到 AL。

OUT: I/O 端口输出。语法: OUT 0x80, AL 将 AL 的值输出到端口 0x80。

3. 目的地址传送指令

LEA: 装入有效地址。例: LEA DX, [SI] 把 SI 的偏移地址存到 DX。

LDS: 传送目标指针, 把指针内容装入 DS。例: LDS SI, [BX] 把段地址:偏移地址存到 DS:SI。

LES: 传送目标指针, 把指针内容装入 ES。例: LES DI, [BX] 把段地址:偏移地址存到 ES:DI。

LFS: 传送目标指针, 把指针内容装入 FS。例: LFS DI, [BX] 把段地址:偏移地址存到 FS:DI。

LGS: 传送目标指针, 把指针内容装入 GS。例: LGS DI, [BX] 把段地址:偏移地址存到 GS:DI。

LSS: 传送目标指针, 把指针内容装入 SS。例: LSS DI, [BX] 把段地址:偏移地址存到 SS:DI。

4. 标志传送指令

LAHF: 标志寄存器传送, 把标志装入 AH。例: LAHF。

SAHF: 标志寄存器传送, 把 AH 内容装入标志寄存器。例: SAHF。

PUSHF: 标志入栈。例: PUSHF。

POPF: 标志出栈。例: POPF。

PUSHD: 32 位标志入栈。例: PUSHD。

POPD: 32 位标志出栈。例: POPD。

二、算术运算指令

1. 加法运算

ADD: 加法。例: ADD AX, BX 将 BX 的值加到 AX。

ADC: 带进位加法。例: ADC AX, 100 将 100 与 AX 的值相加, 考虑进位。

INC: 加 1。例: INC CX 将 CX 的值加 1。

AAA: 加法的 ASCII 码调整。例: MOV AL, 95, ADD AL, 8, AAA 将 AL 的值 95 与 8 相加, 调整为 ASCII 码形式。

DAA: 加法的十进制调整。例: MOV AL, 45, ADD AL, 30, DAA 将 AL 的值 45 与 30 相加, 调整为十进制形式。

2. 减法运算

SUB: 减法。例: SUB DX, AX 将 AX 的值减去 DX。

SBB: 带借位减法。例: SBB BX, 200 将 BX 的值减去 200, 考虑借位。

DEC: 减 1。例: DEC SI 将 SI 的值减 1。

NEG: 求反（以 0 减之）。例: NEG CX 将 CX 的值求反。

CMP: 比较（两操作数作减法，仅修改标志位，不回送结果）。例: CMP AX, BX 比较 AX 和 BX 的值。

3. ASCII 码调整和十进制调整

AAS: 减法的 ASCII 码调整。例: MOV AL, 20, SUB AL, 15, AAS 将 AL 的值 20 与 15 相减，调整为 ASCII 码形式。

DAS: 减法的十进制调整。例: MOV AL, 25, SUB AL, 12, DAS 将 AL 的值 25 与 12 相减，调整为十进制形式。

4. 乘法运算

MUL: 无符号乘法。例: MOV AX, 100, MUL BX 将 AX 的值与 BX 相乘，结果回送到 DX 和 AX。

IMUL: 整数乘法。例: MOV AX, -50, IMUL CX 将 AX 的值与 CX 相乘，结果回送到 DX 和 AX。

AAM: 乘法的 ASCII 码调整。例: MOV AX, 200, DIV BL, AAM 将 AX 的值 200 与 BL 相乘，调整为 ASCII 码形式。

5. 除法运算

DIV: 无符号除法。例: MOV AX, 500, DIV CX 将 AX 的值与 CX 相除，商回送到 AX，余数回送到 DX。

IDIV: 整数除法。例: MOV AX, -800, IDIV DX 将 AX 的值与 DX 相除，商回送到 AX，余数回送到 DX。

AAD: 除法的 ASCII 码调整。例: MOV AX, 350, DIV BX, AAD 将 AX 的值 350 与 BX 相除，调整为 ASCII 码形式。

6. 符号扩展和双字扩展

CBW: 字节转换为字。例: MOV AL, -30, CBW 将 AL 中字节的符号扩展到 AH 中。

CWD: 字转换为双字。例: MOV AX, -200, CWD 将 AX 中的字的符号扩展到 DX 中。

CWDE: 字转换为双字。例: MOV AX, -150, CWDE 将 AX 中的字符号扩展到 EAX 中。

CDQ: 双字扩展。例: MOV EAX, -500, CDQ 将 EAX 中的字的符号扩展到 EDX 中。

三、逻辑运算指令

1. 位运算

AND: 与运算。例: AND AX, BX 将 AX 和 BX 的每一位进行与运算。

OR: 或运算。例: OR CX, DX 将 CX 和 DX 的每一位进行或运算。

XOR: 异或运算。例: XOR SI, DI 将 SI 和 DI 的每一位进行异或运算。

NOT: 取反。例: NOT AX 对 AX 的每一位取反。

TEST: 测试（两操作数作与运算，仅修改标志位，不回送结果）。例: TEST AX, BX 测试 AX 和 BX 的每一位，仅修改标志位。

2. 移位运算

SHL / SAL: 逻辑左移 / 算术左移。例: SHL AX, 1 将 AX 的所有位向左移动 1 位。

SHR / SAR: 逻辑右移 / 算术右移。例: SAR BX, 2 将 BX 的所有位向右移动 2 位。

ROL: 循环左移。例: ROL CX, 3 将 CX 的所有位进行循环左移 3 次。

ROR: 循环右移。例: ROR DX, 4 将 DX 的所有位进行循环右移 4 次。

RCL: 通过进位的循环左移。例: RCL AX, 1 将 AX 的所有位通过进位进行循环左移 1 次。

RCR: 通过进位的循环右移。例: RCR BX, 2 将 BX 的所有位通过进位进行循环右移 2 次。

注意:

以上八种移位指令，其移位次数可达 255 次。

移位一次时，可直接用操作码。如 `SHL AX, 1`。

移位大于 1 次时，由寄存器 CL 给出移位次数。如 `MOV CL, 04, SHL AX, CL`。

位运算示例：

`AND AX, BX` ; $AX = AX \& BX$

`OR CX, DX` ; $CX = CX | DX$

`XOR SI, DI` ; $SI = SI \wedge DI$

`NOT AX` ; $AX = \sim AX$

`TEST AX, BX` ; 测试 AX 和 BX 的位

移位运算示例：

`SHL AX, 1` ; AX 左移 1 位

`SAR BX, 2` ; BX 右移 2 位（算术右移）

`ROL CX, 3` ; CX 循环左移 3 位

`ROR DX, 4` ; DX 循环右移 4 位

`RCL AX, 1` ; AX 通过进位左移 1 位

`RCR BX, 2` ; BX 通过进位右移 2 位

四、串指令

在串指令中，涉及到以下寄存器和标志：

DS:SI: 源串段寄存器和源串变址。

ES:DI: 目标串段寄存器和目标串变址。

CX: 重复次数计数器。

AL/AX: 扫描值。

D 标志: 0 表示重复操作中 SI 和 DI 应自动增量，1 表示应自动减量。

Z 标志: 用来控制扫描或比较操作的结束。

1. 串传送指令

MOVS: 串传送。可以传送字符、字或双字。具体指令包括:

MOVSB: 传送字符。

MOVSW: 传送字。

MOVSD: 传送双字。

2. 串比较指令

CMPS: 串比较。可以比较字符或字。具体指令包括:

CMPSB: 比较字符。

CMPSW: 比较字。

3. 串扫描指令

SCAS: 串扫描。将 AL 或 AX 的内容与目标串作比较, 比较结果反映在标志位。

4. 装入串指令

LODS: 装入串。可以装入字符、字或双字。具体指令包括:

LODSB: 装入字符。

LODSW: 装入字。

LODSD: 装入双字。

5. 保存串指令

STOS: 保存串。是 LODS 的逆过程。

6. 重复前缀

REP: 当 CX/ECX 不等于 0 时重复执行后面的串操作指令。

REPE/REPZ: 当 ZF=1 或比较结果相等, 且 CX/ECX 不等于 0 时重复执行后面的串操作指令。

REPNE/REPNZ: 当 ZF=0 或比较结果不相等, 且 CX/ECX 不等于 0 时重复执行后面的串操作指令。

REPC: 当 CF=1 且 CX/ECX 不等于 0 时重复执行后面的串操作指令。

REPNC: 当 CF=0 且 CX/ECX 不等于 0 时重复执行后面的串操作指令。

串传送示例

MOVS_B ; 传送源串中的一个字符到目标串

MOVSW ; 传送源串中的一个字到目标串

MOVSD ; 传送源串中的一个双字到目标串

串比较示例

CMPS_B ; 比较源串和目标串中的一个字符

CMPSW ; 比较源串和目标串中的一个字

串扫描示例

SCAS_B ; 扫描目标串, 比较 AL 和目标串中的一个字符

装入串示例

LODS_B ; 装入源串中的一个字符到 AL

LODSW ; 装入源串中的一个字到 AX

保存串示例

STOS_B ; 保存 AL 的值到目标串中的一个字符

STOSW ; 保存 AX 的值到目标串中的一个字

重复前缀示例

MOV CX, 5 ; 设置 CX 为 5

REP MOVS ; 重复 5 次将源串传送到目标串

五、程序转移指令

1. 无条件转移指令 (长转移)

JMP: 无条件转移指令。

CALL: 过程调用。

RET/RETF: 过程返回。

2. 条件转移指令 (短转移, -128 到+127 的距离内)

JA/JNBE: 不小于或不等于时转移。

JAE/JNB: 大于或等于转移。

JB/JNAE: 小于转移。

JBE/JNA: 小于或等于转移。

以上四条, 测试无符号整数运算的结果 (标志 C 和 Z)。

JG/JNLE: 大于转移。

JGE/JNL: 大于或等于转移。

JL/JNGE: 小于转移。

JLE/JNG: 小于或等于转移。

以上四条, 测试带符号整数运算的结果 (标志 S, O 和 Z)。

JE/JZ: 等于转移。

JNE/JNZ: 不等于时转移。

JC: 有进位时转移。

JNC: 无进位时转移。

JNO: 不溢出时转移。

JNP/JPO: 奇偶性为奇数时转移。

JNS: 符号位为 "0" 时转移。

JO: 溢出转移。

JP/JPE: 奇偶性为偶数时转移。

JS: 符号位为 "1" 时转移。

3. 循环控制指令 (短转移)

LOOP: CX 不为零时循环。

LOOPE/LOOPZ: CX 不为零且标志 Z=1 时循环。

LOOPNE/LOOPNZ: CX 不为零且标志 Z=0 时循环。

JCXZ: CX 为零时转移。

JECXZ: ECX 为零时转移。

4. 中断指令

INT: 中断指令。

INTO: 溢出中断。

IRET: 中断返回。

5. 处理器控制指令

HLT: 处理器暂停，直到出现中断或复位信号才继续。

WAIT: 当芯片引线 TEST 为高电平时使 CPU 进入等待状态。

ESC: 转换到外处理器。

LOCK: 封锁总线。

NOP: 空操作。

STC: 置进位标志位。

CLC: 清进位标志位。

CMC: 进位标志取反。

STD: 置方向标志位。

CLD: 清方向标志位。

STI: 置中断允许位。

CLI: 清中断允许位。

无条件转移指令:

JMP Label ; 无条件跳转到 Label

CALL Subroutine ; 调用子程序

RET ; 返回

条件转移指令:

JZ Target ; 如果标志 ZF=1, 则跳转到 Target

JG GreaterLabel ; 如果 SF=OF=0 且 ZF=0, 则跳转到 GreaterLabel

循环控制指令:

LOOP LoopLabel ; CX 不为零时循环, 循环到 LoopLabel

JCXZ ZeroLabel ; 如果 CX 为零, 则跳转到 ZeroLabel

中断指令:

INT 21h ; 触发 21h 中断

IRET ; 中断返回

处理器控制指令:

HLT ; 处理器暂停

NOP ; 空操作

STI ; 置中断允许位

七、处理机控制指令：标志处理指令

CLC: 进位位置 0 指令。将进位标志位 CF 置为 0。

CMC: 进位位求反指令。将进位标志位 CF 取反。

STC: 进位位置为 1 指令。将进位标志位 CF 置为 1。

CLD: 方向标志置 0 指令。将方向标志位 DF 置为 0。

STD: 方向标志位置 1 指令。将方向标志位 DF 置为 1。

CLI: 中断标志置 0 指令。将中断标志位 IF 置为 0。

STI: 中断标志置 1 指令。将中断标志位 IF 置为 1。

NOP: 无操作。不执行任何操作，用于占位或调整指令顺序。

HLT: 停机。处理器暂停执行指令，直到出现中断或复位信号才继续。

WAIT: 等待。当芯片引线 TEST 为高电平时使 CPU 进入等待状态。

ESC: 换码。用于向外部处理器传送数据。

LOCK: 封锁。用于在多处理器环境中封锁总线，确保原子操作。

补充：

浮点运算指令集

一. 控制指令

(带 9B 的控制指令前缀 F 变为 FN 时浮点不检查，机器码去掉 9B)

FINIT: 初始化浮点部件。用于初始化 80387 或 80487 浮点协处理器。

机器码: DB E3

FCLEX: 清除异常。清除浮点异常标志。

机器码: DB E2

FDISI: 浮点检查禁止中断。禁止浮点指令引发中断。

机器码: DB E1

FENI: 浮点检查禁止中断二。允许浮点指令引发中断。

机器码: DB E0

WAIT / FWAIT: 同步 CPU 和 FPU。等待浮点操作完成，同步 FPU 和 CPU。

机器码: 9B (WAIT), D9 D0 (FWAIT)

FNOP: 无操作。不执行任何浮点操作。

机器码: DA E9

FXCH: 交换 ST(0)和 ST(1)。交换浮点寄存器堆栈中的两个栈顶元素。

机器码: D9 C9

FXCH ST(i): 交换 ST(0)和 ST(i)。交换浮点寄存器堆栈中的 ST(0)和 ST(i)。

机器码: D9 C1iii (iii 表示寄存器编号)

FSTSW ax: 状态字到 ax。将浮点状态字传送到寄存器 ax。

机器码: 9B DF E0

FSTSW word ptr mem: 状态字到 mem。将浮点状态字传送到内存地址 mem。

机器码: 9B DD mm11mmm

FLDCW word ptr mem: mem 到状态字。加载控制字 (CW) 从内存地址 mem 到 FPU。

机器码: D9 mm101mmm

FSTCW word ptr mem: 控制字到 mem。将浮点控制字传送到内存地址 mem。

机器码: 9B D9 mm111mmm

FLDENV word ptr mem: mem 到全环境。加载整个浮点环境从内存地址 mem 到 FPU。

机器码: D9 mm100mmm

FSTENV word ptr mem: 全环境到 mem。将整个浮点环境传送到内存地址 mem。

机器码: 9B D9 mm110mmm

FRSTOR word ptr mem: mem 到 FPU 状态。从内存地址 mem 恢复 FPU 状态。

机器码: DD mm100mmm

FSAVE word ptr mem: FPU 状态到 mem。保存 FPU 状态到内存地址 mem。

机器码: 9B DD mm110mmm

FFREE ST(i): 标志 ST(i)未使用。标记浮点寄存器 ST(i)为空闲状态。

机器码: DD C0iii (iii 表示寄存器编号)

FDECSTP: 减少栈指针。减少浮点寄存器栈指针。

机器码: D9 F6

FINCSTP: 增加栈指针。增加浮点寄存器栈指针。

机器码: D9 F7

FSETPM: 浮点设置保护。设置浮点操作的保护模式。

机器码: DB E4

FINIT ; 初始化浮点部件

FCLEX ; 清除异常

FDISI ; 浮点检查禁止中断

FENI ; 浮点检查禁止中断二

WAIT ; 同步 CPU 和 FPU

FNOP ; 无操作

FXCH ; 交换 ST(0)和 ST(1)

FSTSW ax ; 将状态字传送到寄存器 ax

FSTSW word ptr mem ; 将状态字传送到内存地址 mem

FLDCW word ptr mem ; 从内存地址 mem 加载控制字到 FPU

FSTCW word ptr mem ; 将浮点控制字传送到内存地址 mem

FLDENV word ptr mem ; 从内存地址 mem 加载整个浮点环境到 FPU

FSTENV word ptr mem ; 将整个浮点环境传送到内存地址 mem

FRSTOR word ptr mem ; 从内存地址 mem 恢复 FPU 状态

FSAVE word ptr mem ; 将 FPU 状态保存到内存地址 mem

FFREE ST(1) ; 标志 ST(1)未使用

FDECSTP ; 减少栈指针

FINCSTP ; 增加栈指针

FSETPM ; 设置浮点操作的保护模式

二、数据传送指令

FLDZ: 将 0.0 装入 ST(0)。将浮点数 0.0 加载到浮点寄存器 ST(0)。

机器码: D9 EE

FLD1: 将 1.0 装入 ST(0)。将浮点数 1.0 加载到浮点寄存器 ST(0)。

机器码: D9 E8

FLDPI: 将 π 装入 ST(0)。将 π 加载到浮点寄存器 ST(0)。

机器码: D9 EB

FLDL2T: 将 $\ln 10 / \ln 2$ 装入 ST(0)。将 $\ln 10 / \ln 2$ 加载到浮点寄存器 ST(0)。

机器码: D9 E9

FLDL2E: 将 $1 / \ln 2$ 装入 ST(0)。将 $1 / \ln 2$ 加载到浮点寄存器 ST(0)。

机器码: D9 EA

FLDLG2: 将 $\ln 2/\ln 10$ 装入 ST(0)。将 $\ln 2/\ln 10$ 加载到浮点寄存器 ST(0)。

机器码: D9 EC

FLDLN2: 将 $\ln 2$ 装入 ST(0)。将 $\ln 2$ 加载到浮点寄存器 ST(0)。

机器码: D9 ED

FLD real4 ptr mem: 装入 mem 的单精度浮点数。将单精度浮点数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: D9 mm000mmm

FLD real8 ptr mem: 装入 mem 的双精度浮点数。将双精度浮点数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DD mm000mmm

FLD real10 ptr mem: 装入 mem 的十字节浮点数。将十字节浮点数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DB mm101mmm

FILD word ptr mem: 装入 mem 的二字节整数。将二字节整数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DF mm000mmm

FILD dword ptr mem: 装入 mem 的四字节整数。将四字节整数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DB mm000mmm

FILD qword ptr mem: 装入 mem 的八字节整数。将八字节整数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DF mm101mmm

FBLD tbyte ptr mem: 装入 mem 的十字节 BCD 数。将十字节 BCD 数从内存地址 mem 加载到浮点寄存器 ST(0)。

机器码: DF mm100mmm

FST real4 ptr mem: 保存单精度浮点数到 mem。将浮点寄存器 ST(0)的单精度浮点数保存到内存地址 mem。

机器码: D9 mm010mmm

FST real8 ptr mem: 保存双精度浮点数到 mem。将浮点寄存器 ST(0)的双精度浮点数保存到内存地址 mem。

机器码: DD mm010mmm

FIST word ptr mem: 保存二字节整数到 mem。将浮点寄存器 ST(0)的整数部分保存为二字节整数到内存地址 mem。

机器码: DF mm010mmm

FIST dword ptr mem: 保存四字节整数到 mem。将浮点寄存器 ST(0)的整数部分保存为四字节整数到内存地址 mem。

机器码: DB mm010mmm

FSTP real4 ptr mem: 保存单精度浮点数到 mem 并出栈。将浮点寄存器 ST(0)的单精度浮点数保存到内存地址 mem，并出栈。

机器码: D9 mm011mmm

FSTP real8 ptr mem: 保存双精度浮点数到 mem 并出栈。将浮点寄存器 ST(0)的双精度浮点数保存到内存地址 mem，并出栈。

机器码: DD mm011mmm

FSTP real10 ptr mem: 保存十字节浮点数到 mem 并出栈。将浮点寄存器 ST(0)的十字节浮点数保存到内存地址 mem, 并出栈。

机器码: DB mm111mmm

FISTP word ptr mem: 保存二字节整数到 mem 并出栈。将浮点寄存器 ST(0)的整数部分保存为二字节整数到内存地址 mem, 并出栈。

机器码: DF mm011mmm

FISTP dword ptr mem: 保存四字节整数到 mem 并出栈。将浮点寄存器 ST(0)的整数部分保存为四字节整数到内存地址 mem, 并出栈。

机器码: DB mm011mmm

FISTP qword ptr mem: 保存八字节整数到 mem 并出栈。将浮点寄存器 ST(0)的整数部分保存为八字节整数到内存地址 mem, 并出栈。

机器码: DF mm111mmm

FBSTP tbyte ptr mem: 保存十字节 BCD 数到 mem 并出栈。将浮点寄存器 ST(0)的十字节 BCD 数保存到内存地址 mem, 并出栈。

机器码: DF mm110mmm

FCMOVB ST(0),ST(i): 当 $ST(0) < ST(i)$ 时传送。如果 ST(0) 小于 ST(i), 则将 ST(0) 的值传送到 ST(i)。

机器码: DA C0iii (iii 表示寄存器编号)

FCMOVBE ST(0),ST(i): 当 $ST(0) \leq ST(i)$ 时传送。如果 ST(0) 小于或等于 ST(i), 则将 ST(0) 的值传送到 ST(i)。

机器码: DA D0iii (iii 表示寄存器编号)

FCMOVE ST(0),ST(i): 当 $ST(0) = ST(i)$ 时传送。如果 ST(0) 等于 ST(i), 则将 ST(0) 的值传送到 ST(i)。

机器码: DA C1iii (iii 表示寄存器编号)

FCMOVNB ST(0),ST(i): 当 ST(0) $\geq$ ST(i)时传送。如果 ST(0)大于或等于 ST(i), 则将 ST(0)的值传送到 ST(i)。

机器码: DB C0iii (iii 表示寄存器编号)

FCMOVNBE ST(0),ST(i): 当 ST(0) > ST(i)时传送。如果 ST(0)大于 ST(i), 则将 ST(0)的值传送到 ST(i)。

机器码: DB D0iii (iii 表示寄存器编号)

FCMOVNE ST(0),ST(i): 当 ST(0) $\neq$ ST(i)时传送。如果 ST(0)不等于 ST(i), 则将 ST(0)的值传送到 ST(i)。

机器码: DB C1iii (iii 表示寄存器编号)

FCMOVNU ST(0),ST(i): 当 ST(0) 有序时传送。如果 ST(0)和 ST(i)都是有序数, 将 ST(0)的值传送到 ST(i)。

机器码: DB D1iii (iii 表示寄存器编号)

FCMOVU ST(0),ST(i): 当 ST(0) 无序时传送。如果 ST(0)和 ST(i)都是无序数, 将 ST(0)的值传送到 ST(i)。

机器码: DA D1iii (iii 表示寄存器编号)

三、比较指令

FCOM: 比较 ST(0)和 ST(1)。比较浮点寄存器 ST(0)和 ST(1)中的值。

机器码: D8 D1

FCOMI ST(0),ST(i): 比较 ST(0)和 ST(i), 结果影响 EFLAGS 寄存器。比较浮点寄存器 ST(0)和 ST(i)中的值。

机器码: DB F0iii (iii 表示寄存器编号)

FCOMIP ST(0),ST(i): 比较 ST(0)和 ST(i)并出栈, 结果影响 EFLAGS 寄存器。比较浮点寄存器 ST(0)和 ST(i)中的值, 并将 ST(0)出栈。

机器码: DF F0iii (iii 表示寄存器编号)

FCOM real4 ptr mem: 比较 ST(0)和实数 mem。比较浮点寄存器 ST(0)和内存地址 mem 中的单精度浮点数。

机器码: D8 mm010mmm

FCOM real8 ptr mem: 比较 ST(0)和实数 mem。比较浮点寄存器 ST(0)和内存地址 mem 中的双精度浮点数。

机器码: DC mm010mmm

FICOM word ptr mem: 比较 ST(0)和整数 mem。比较浮点寄存器 ST(0)和内存地址 mem 中的二字节整数。

机器码: DE mm010mmm

FICOM dword ptr mem: 比较 ST(0)和整数 mem。比较浮点寄存器 ST(0)和内存地址 mem 中的四字节整数。

机器码: DA mm010mmm

FICOMP word ptr mem: 比较 ST(0)和整数 mem 并出栈。比较浮点寄存器 ST(0)和内存地址 mem 中的二字节整数, 并将 ST(0)出栈。

机器码: DE mm011mmm

FICOMP dword ptr mem: 比较 ST(0)和整数 mem 并出栈。比较浮点寄存器 ST(0)和内存地址 mem 中的四字节整数, 并将 ST(0)出栈。

机器码: DA mm011mmm

FTST: 比较 ST(0)和 0。比较浮点寄存器 ST(0)和常数 0。

机器码: D9 E4

FUCOM ST(i): 比较 ST(0)和 ST(i), 结果影响 EFLAGS 寄存器。比较浮点寄存器 ST(0)和 ST(i)中

的值。

机器码: DD E0iii (iii 表示寄存器编号)

FUCOMP ST(i): 比较 ST(0)和 ST(i)并出栈, 结果影响 EFLAGS 寄存器。比较浮点寄存器 ST(0)和 ST(i)中的值, 并将 ST(0)出栈。

机器码: DD E1iii (iii 表示寄存器编号)

FUCOMPP: 比较 ST(0)和 ST(1)并二次出栈, 结果影响 EFLAGS 寄存器。比较浮点寄存器 ST(0)和 ST(1)中的值, 并将 ST(0)和 ST(1)出栈。

机器码: DA E9

FXAM: 规格类型指令, 检查 ST(0)中的浮点数的类型。设置 EFLAGS 寄存器的标志以指示 ST(0)中的值的类型。

机器码: D9 E5

四.运算指令

FADD

描述: 将目的操作数与来源操作数相加, 并将结果存入目的操作数。

例子: FADD DST, SRC (将目的操作数 DST 与来源操作数 SRC 相加)

FADDP ST(i), ST

描述: 将目的操作数加上 ST 缓存器的值, 然后弹出 ST 缓存器。目的操作数必须是堆栈缓存器的其中之一。

例子: FADDP ST(1), ST (将 ST(1)的值加到 ST 上, 并弹出 ST(1))

FIADD

描述: 将 ST 的值加上来源操作数, 然后存入 ST 缓存器。来源操作数必须是字组整数或短整数形态的变量。

例子: FIADD INT_VAR (将整数变量 INT_VAR 的值加到 ST 上)

FSUB

描述: 减法操作。

例子: FSUB DST, SRC (将目的操作数 DST 减去来源操作数 SRC)

FSUBP

描述: 将目的操作数减去 ST 缓存器的值, 然后弹出 ST 缓存器。

例子: FSUBP ST(2), ST (将 ST(2)的值减去 ST 上, 并弹出 ST(2))

FSUBR

描述: 将 ST 的值减去目的操作数, 结果存入目的操作数。

例子: FSUBR DST, SRC (将来源操作数 SRC 减去目的操作数 DST, 并将结果存入 DST)

FSUBRP

描述: 将 ST 的值减去目的操作数, 然后弹出 ST 缓存器。

例子: FSUBRP ST(3), ST (将 ST(3)的值减去 ST 上, 并弹出 ST(3))

FISUB

描述: 将 ST 的值减去来源操作数, 结果存入 ST。

例子: FISUB INT_VAR (将整数变量 INT_VAR 的值减去 ST)

FISUBR

描述: 将来源操作数减去 ST 的值, 结果存入 ST。

例子: FISUBR INT_VAR (将 ST 的值减去整数变量 INT_VAR)

FMUL

描述: 乘法操作。

例子: FMUL DST, SRC (将目的操作数 DST 与来源操作数 SRC 相乘)

FMULP

描述: 将目的操作数乘以 ST 的值, 然后弹出 ST 缓存器。

例子: FMULP ST(4), ST (将 ST(4)的值乘以 ST 上, 并弹出 ST(4))

FIMUL

描述: 将 ST 的值乘以来源操作数, 结果存入 ST。

例子: FIMUL INT_VAR (将 ST 的值乘以整数变量 INT_VAR)

FDIV

描述: 除法操作。

例子: FDIV DST, SRC (将目的操作数 DST 除以来源操作数 SRC)

FDIVP

描述: 将目的操作数除以 ST 的值, 然后弹出 ST 缓存器。

例子: FDIVP ST(5), ST (将 ST(5)的值除以 ST 上, 并弹出 ST(5))

FDIVR

描述: 将 ST 的值除以目的操作数, 结果存入目的操作数。

例子: FDIVR DST, SRC (将来源操作数 SRC 除以目的操作数 DST, 并将结果存入 DST)

FDIVRP

描述: 将 ST 的值除以目的操作数, 然后弹出 ST 缓存器。

例子: FDIVRP ST(6), ST (将 ST(6)的值除以 ST 上, 并弹出 ST(6))

FIDIV

描述: 将 ST 的值除以来源操作数, 结果存入 ST。

例子: FIDIV INT_VAR (将 ST 的值除以整数变量 INT_VAR)

FIDIVR

描述: 将来源操作数除以 ST 的值, 结果存入 ST。

例子: FIDIVR INT_VAR (将整数变量 INT_VAR 除以 ST 的值, 并将结果存入 ST)

FCFS

描述: 改变 ST 的正负值。

例子: FCHS (改变 ST 的正负值)

FABS

描述: 取 ST 的绝对值。

例子: FABS (取 ST 的绝对值)

FSQRT

描述: 将 ST 的值开平方后存回 ST。

例子: FSQRT (将 ST 的值开平方后存回 ST)

FSCALE

描述: 计算 ST 乘以 2 的 ST(1)次方的值, 结果存入 ST。ST(1)必须是在 -32768 到 32768 范围内的整数。

例子: FSCALE (计算 ST 乘以 2 的 ST(1)次方的值)

FRNDINT

描述: 将 ST 的数值舍入成整数, 舍入方式由 FPU 的控制字组中的 RC 位决定。

RC 舍入控制:

00: 四舍五入

01: 向负无限大舍入

10: 向正无限大舍入

11: 向零舍去

例子: FRNDINT (将 ST 的数值四舍五入为整数)

本文件由 **公众号: 数基培文** 整理, 数学/计算机/文学内容, 欢迎关注, 获取更多大学学习资料。